Informe de solución S05: revocación de cookies al cerrar sesión

Fecha: 9 de noviembre de 2026. Repositorio: kevinxonium153/DiversifyDelivery. Rama: beta. Base de la corrección: 39de436. Estado: corregido y validado en local; despliegue en producción pendiente.

Problema y causa

Antes, POST /api/auth/logout solo eliminaba la cookie del navegador que solicitaba salir. Una copia válida conservaba las credenciales firmadas y podía seguir accediendo a la API. La auditoría original observó HTTP 200 con esa copia después de recibir HTTP 204 en logout. La caché de estado del usuario no distinguía las sesiones individuales y no podía revocar una cookie concreta.

El fallo requería haber obtenido una cookie válida previamente; esta revisión no demostró una vía para robarla.

Solución implementada

Se añadió UserSessions en SQLite. Cada sesión contiene la huella SHA-256 de un identificador aleatorio de 256 bits, el ID del usuario y su vencimiento en UTC. El identificador original viaja como claim session_id dentro de la cookie protegida por ASP.NET Core. No se guarda la cookie ni el identificador original en la tabla.

Antes de aceptar cualquier petición autenticada se consulta la fila correspondiente, sin caché de revocación. Deben coincidir sesión y usuario, y el registro debe seguir vigente. Si falta o venció, se rechaza la identidad: un endpoint protegido responde 401 y /api/auth/me responde 204 como visitante.

Logout borra esa fila antes de emitir la respuesta que elimina la cookie del navegador. Todas las copias de esa sesión dejan de servir para nuevas solicitudes, sin esperar al vencimiento del ticket ni a la caché de usuario. La revocación persiste al reiniciar porque reside en SQLite. Otra sesión independiente del mismo usuario conserva su registro y sigue funcionando.

Los inicios de sesión explícitos —login, registro, cambio de perfil/contraseña y recuperación de contraseña— crean un identificador nuevo y revocan el de la sesión actual, si existía. Esto evita reutilizar la misma cookie después de actualizar sus credenciales. La renovación automática de claims conserva el identificador existente: no puede crear una sesión nueva después de haber sido revocada.

La vigencia usa el mismo plazo de 30 días que el ticket. Se renueva cuando se renuevan los claims o cuando queda la mitad o menos de la vigencia del registro. La actualización solo afecta filas que todavía existen y están vigentes; una petición concurrente no puede recrear una fila eliminada por logout. Las filas caducadas se limpian al iniciar sesión.

Se mantienen los controles previos de contraseña, rol y estado de cuenta. La consulta de sesión no usa caché; la consulta del usuario conserva la caché previa de 30 segundos y las invalidaciones explícitas existentes.

Archivos y compatibilidad

- Core/Entities.cs: entidad UserSession.

- Core/DeliveryDbContext.cs: tabla, índices y relación con usuario.

- Core/Security.cs: inicio con rotación, revocación, validación y renovación de sesiones.

- Modules/AuthEndpoints.cs: login/registro/perfil usan el nuevo inicio; logout revoca antes de borrar la cookie.

- Modules/PasswordResetEndpoints.cs: recuperación inicia una sesión nueva mediante el mismo mecanismo.

- Program.cs: duración de la cookie centralizada con la de la sesión.

- tests/seguridad-sesiones.cjs y CI: regresión automatizada con procesos, puerto y base propios.

SchemaUpgrades ya crea tablas nuevas a partir del modelo. Por eso no se requiere renombrar ni borrar tablas existentes ni añadir columnas manualmente. La prueba quitó únicamente UserSessions de SU base desechable, reinició la app y confirmó que se recreó conservando el usuario y sus credenciales.

Las cookies anteriores a esta versión no incluyen session_id y se rechazan; el usuario debe iniciar sesión de nuevo. Se conserva el nombre de cookie, sus banderas HttpOnly/SameSite y Secure según el entorno, y las llaves de protección existentes. No se añadieron paquetes ni secretos.

Evidencia de validación

Suite tests/seguridad-sesiones.cjs: 71 comprobaciones correctas.

- Se accede con una cookie válida para calentar también la caché del usuario; logout devuelve 204.

- Diez reintentos con la copia exacta de esa cookie reciben 401 en el endpoint protegido.

- /auth/me recibe esa cookie y devuelve 204; repetir logout también devuelve 204.

- Otra sesión independiente del mismo usuario sigue recibiendo 200.

- Editar el perfil rota la cookie: la anterior devuelve 401 y la nueva 200.

- Tras reiniciar la aplicación, la sesión independiente sigue válida y la revocada continúa rechazada.

- Al aproximar el vencimiento de la fila de prueba se comprueba la renovación de su vigencia.

- Doce solicitudes concurrentes con renovación y logout no recrean la sesión: al acabar queda cero filas para ese usuario y el siguiente intento recibe 401, también después de otro reinicio.

- Cambiar la contraseña invalida las cookies anteriores, incluida la de otro equipo, y deja válida la sesión nueva.

- Suspender al usuario impide el acceso con su sesión.

- Una sesión vencida en la base se rechaza aunque el ticket del navegador aún no haya vencido.

- Iniciar sesión limpia las filas caducadas; recrear la tabla en una base existente no pierde el usuario.

Las fechas y la eliminación de tabla se simularon exclusivamente en la base aislada de la suite mediante python3; no existen endpoints de depuración para hacerlo. Los identificadores/cookies no se imprimen en los resultados.

Otras verificaciones aprobadas:

- Compilación .NET 10 con -warnaserror, sin errores ni advertencias.

- JavaScript y ShellCheck.

- 580 comprobaciones de S01/S02/S03: permisos, toma atómica y máximo de pedidos pendientes.

- Recorrido completo en Chromium: registro, pago simulado, pedido, preparación, toma y entrega con PIN.

- Recuperación de contraseña en Chromium: enlace de un solo uso, nueva sesión y cierre de la sesión del otro equipo.

Todas las pruebas usaron datos demo locales. Los correos de prueba se guardaron como .eml; no se envió correo externo ni se atacó producción. La evidencia de 200 tras logout corresponde a la auditoría anterior; la suite nueva confirma el rechazo de la cookie con la corrección.

Reproducción

Desde la raíz del repositorio:

```
dotnet build src/Diversify.Delivery -warnaserror
```

```
node tests/seguridad-sesiones.cjs
```

Requiere Node 22+, dotnet y python3. La suite arranca y detiene solo sus propios procesos y conserva sus datos demo en test-results/sesiones-*, ignorado por Git. El resultado esperado es 71 comprobaciones correctas. El límite de login se eleva solo dentro de esa instancia de pruebas para comprobar varios equipos sin esperar; la configuración normal de la app no cambia. La suite ya está incorporada al CI.

Despliegue, límites y pendientes

1. Respaldar la base y revisar el diff en beta siguiendo el flujo habitual del repositorio.

2. Actualizar y reiniciar todas las instancias de la aplicación. No dejar servidores de la versión anterior atendiendo peticiones: estos no conocen la revocación y todavía aceptarían sus cookies antiguas.

3. Mantener las llaves de Data Protection y la base SQLite compartidas por las instancias que deban reconocer las mismas sesiones. Las copias independientes de la base no comparten el estado de revocación.

4. Confirmar la creación de UserSessions y comprobar login, logout y replay con cuentas de prueba. Avisar de que las sesiones anteriores al cambio deberán iniciar sesión de nuevo.

Una petición que ya fue autenticada antes de logout puede terminar su trabajo; logout no cancela operaciones en vuelo. En la prueba concurrente esas respuestas podían ser 200 o 401, pero toda solicitud posterior al cierre se rechaza y no se recrea la sesión. La función cierra la sesión actual, sin añadir una pantalla de “cerrar todos los equipos”. No se trata de una corrección CSRF: ese hallazgo S04 sigue pendiente.

También siguen pendientes S06 (transferencias sin límite), S07 (método de pago inválido), S08 (validación de imágenes) y los asuntos de operación/producto de notas/Pendientes.md. El push a beta no despliega producción ni demuestra que el CI remoto haya terminado.